Notebook : Add Cell Type Annotation – Manual Cluster Labeling

This notebook adds a `cell_type_annotation` column to the preprocessed dataset (`adata_pp.h5ad`) from `single_cell_pipeline.ipynb`.

This annotation step is **manual** and required for downstream pseudobulk differential expression analysis in `translation_to_R.ipynb`.

Pipeline Overview:

- Setup and Configuration
- Load Preprocessed Data
- Define Annotation Mapping (Leiden cluster → Macro cell type)
- Apply Mapping and Validation
- Save Annotated Dataset
- Summary

Note: This notebook provides the structure for annotation. The actual mapping (cluster → cell type) must be filled in manually based on marker gene analysis and biological knowledge.

Setup and Configuration

Environment Setup

Load required libraries and configure working directories for reproducible analysis.

In [1]:

```
import os
import numpy as np
import pandas as pd
import anndata as ad
import scanpy as sc
from pathlib import Path

import warnings
warnings.filterwarnings("ignore")

# Set random seed for reproducibility
np.random.seed(42)

# PROJECT_ROOT = "/Users/elodiehusson/Desktop/AD & PD" # Elodie
```

```
# PROJECT_ROOT = "C:/Users/yarad/Desktop/x/Masters/Master BMC -
Sorbonne/M2/Single Cell/Project/Coding Project" # Yara
PROJECT_ROOT = "C:/Z/AIDA_transcriptomics_project/transcriptomics-code"  #
Laïla

# Define directory structure
DIRS = {
    "DATA":    os.path.join(os.path.dirname(PROJECT_ROOT), "data"),
    "TMP":     os.path.join(PROJECT_ROOT, "tmp_cache")
}

# Create directories if they don't exist
for path in DIRS.values():
    os.makedirs(path, exist_ok=True)

os.chdir(PROJECT_ROOT)

# Define log function for pipeline progress
def log(message):
    """Display pipeline step messages"""
    print(f"📋 {message}")

log(f"Environment loaded. Working directory: {os.getcwd()}")
```

```
📋 Environment loaded. Working directory:
C:\Z\AIDA_transcriptomics_project\transcriptomics-code
```

Load Preprocessed Data

Load Preprocessed Dataset

Load the fully preprocessed AnnData object from `single_cell_pipeline.ipynb` and validate cluster information.

In [2]:

```
# Load the preprocessed dataset (output from single_cell_pipeline.ipynb)
dataset_path = os.path.join(DIRS["DATA"], "adata_pp.h5ad")
adata = sc.read_h5ad(dataset_path)

log(f"Preprocessed dataset loaded: {dataset_path}")
log(f"Dataset dimensions: {adata.shape[0]:,} cells × {adata.shape[1]:,}
genes")
```

```
📋 Preprocessed dataset loaded:
C:/Z/AIDA_transcriptomics_project\data\adata_pp.h5ad
📋 Dataset dimensions: 62,800 cells × 2,000 genes
```

Validation and Cluster Overview

Verify the presence of Leiden clustering and display cluster distribution.

In [3]:

```
# Check presence of leiden column
if 'leiden' not in adata.obs.columns:
    raise ValueError("❌ Missing required column 'leiden' in adata.obs.
Please run single_cell_pipeline.ipynb first.")

log("✅ Leiden clustering found in adata.obs")

# Display cluster summary
n_clusters = adata.obs['leiden'].nunique()
print("\n" + "="*70)
print("📊 CLUSTER SUMMARY")
print("="*70)
print(f"Number of cells: {adata.n_obs:,}")
print(f"Number of genes: {adata.n_vars:,}")
print(f"Number of Leiden clusters: {n_clusters}")
print(f"\nCluster distribution (cells per cluster):")
print(adata.obs['leiden'].value_counts().sort_index())
print("="*70 + "\n")
```

```
📋 ✅ Leiden clustering found in adata.obs

======================================================================
📊 CLUSTER SUMMARY
======================================================================
Number of cells: 62,800
Number of genes: 2,000
Number of Leiden clusters: 23

Cluster distribution (cells per cluster):
leiden
0     8600
1     7611
2     7077
3     5421
4     4752
5     4482
6     3535
7     3118
8     3078
9     3048
10    2984
11    2197
12    1943
13     782
14     740
15     685
```

```
16     663
17     579
18     393
19     372
20     356
21     291
22      93
Name: count, dtype: int64
======================================================================
```

Define Annotation Mapping

Manual Annotation Table

⚠ IMPORTANT: MANUAL STEP REQUIRED ⚠

This step requires **manual annotation** based on:

- Marker gene analysis from `single_cell_pipeline.ipynb`
- Biological knowledge of brain cell types
- Literature references (e.g., PanglaoDB, CellMarker, original publication)

Instructions:

1. Review the marker genes for each Leiden cluster (exported in `marker_genes_leiden.csv` or displayed in `single_cell_pipeline.ipynb`)
2. Assign a **macro cell type** to each cluster in the mapping dictionary below
3. Use consistent naming conventions (e.g., "Excitatory neuron", "Astrocyte", "Microglia")
4. All clusters must be annotated (no missing keys)

Reference cell types for human prefrontal cortex:

- Neurons: Excitatory neuron, Inhibitory neuron
- Glia: Astrocyte, Oligodendrocyte, OPC (Oligodendrocyte Precursor Cell)
- Immune: Microglia
- Vascular: Endothelial, Pericyte, Smooth muscle cell

In [4]:

```
# ANNOTATION MAPPING: Leiden cluster → Macro cell type
#  FILL IN MANUALLY BASED ON MARKER GENE ANALYSIS

#
==========================================================================
==
# MAPPING FINAL VALIDÉ : 8 MACRO-TYPES (Consensus)
```

```
#
==========================================================================
==
# Décision : Granularité Astrocytes + Robustesse Microglia
# Validation : Tous les macro-types ont ≥5 donneurs avec ≥20 cellules
#
==========================================================================
==

mapping = {
    # ------------------------------------------------------------------
----
    # NEURONES EXCITATEURS (10 clusters)
    # Total : ~22,800 cellules | 17 donneurs
    # ------------------------------------------------------------------
----
    "0": "Excitatory neuron",
    "2": "Excitatory neuron",
    "8": "Excitatory neuron",
    "11": "Excitatory neuron",
    "13": "Excitatory neuron",
    "16": "Excitatory neuron",
    "19": "Excitatory neuron",

    # ------------------------------------------------------------------
----
    # NEURONES INHIBITEURS (4 clusters)
    # Total : ~13,000 cellules | 17 donneurs
    # Sous-types : PVALB (Cl.1), SST (Cl.7), VIP (Cl.12), LAMP5 (Cl.20)
    # ------------------------------------------------------------------
----
    "1": "Inhibitory neuron",
    "7": "Inhibitory neuron",
    "12": "Inhibitory neuron",
    "20": "Inhibitory neuron",

    # ------------------------------------------------------------------
----
    # OLIGODENDROCYTES / OPC (3 clusters)
    # Total : ~9,700 cellules | 17 donneurs
    # ------------------------------------------------------------------
----
    "3": "Oligodendrocyte/OPC",
    "6": "Oligodendrocyte/OPC",
    "14": "Oligodendrocyte/OPC",

    # ------------------------------------------------------------------
----
    # ASTROCYTES - SÉPARATION (2 clusters)
```

```
    # Décision : Finesse biologique viable pour AD/PD
    # ------------------------------------------------------------------------
----
    "4": "Astrocyte Homeostatic",  # 4,752 cellules | 9 donneurs ≥20
cellules
    "15": "Astrocyte Reactive",    # 685 cellules   | 7 donneurs ≥20
cellules

    # ------------------------------------------------------------------------
----
    # MICROGLIA - REGROUPEMENT (2 clusters)
    # Décision : Robustesse statistique (évite artefact Cluster 18)
    # ------------------------------------------------------------------------
----
    "5": "Microglia",              # Homeostatic : 4,482 cellules | 14
donneurs
    "18": "Microglia",             # DAM/Activated : 393 cellules | 1
donneur
                                   # → Regroupé pour éviter catégorie vide
en DGE

    # ------------------------------------------------------------------------
----
    # SUPPORT / VASCULAR / IMMUNE (3 clusters)
    # Total : ~6,600 cellules | 17 donneurs
    # Composition : Endothéliales (Cl.9), Murales (Cl.10), Lymphocytes T
(Cl.17)
    # ------------------------------------------------------------------------
----
    "9": "Support/Vascular/Immune",
    "10": "Support/Vascular/Immune",
    "17": "Support/Vascular/Immune",

    # ------------------------------------------------------------------------
----
    # EXCLUSION (2 clusters)
    # Total : ~400 cellules (0.6% du dataset)
    # → Seront supprimés après application du mapping
    # ------------------------------------------------------------------------
----
    "21": "Unknown/Low Quality",
    "22": "Unknown/Low Quality"
}

log(f"Manual Annotation mapping defined for {len(mapping)} clusters")
```

```
📋 Manual Annotation mapping defined for 23 clusters
```

Apply Mapping and Validation

Validate Mapping Completeness

Ensure all Leiden clusters present in the data are mapped to a cell type.

In [5]:

```
# Get unique Leiden clusters in the data
clusters_in_data = set(adata.obs['leiden'].cat.categories.astype(str))
clusters_in_mapping = set(mapping.keys())

# Check for missing clusters in mapping
missing_clusters = clusters_in_data - clusters_in_mapping
if missing_clusters:
    raise ValueError(
        f"❌ ERROR: The following clusters are present in the data but
missing from the mapping:\n"
        f"   {sorted(missing_clusters)}\n"
        f"   Please add these clusters to the mapping dictionary."
    )

# Check for extra clusters in mapping (warning only)
extra_clusters = clusters_in_mapping - clusters_in_data
if extra_clusters:
    log(f"⚠️  WARNING: The following clusters are in the mapping but not in
the data: {sorted(extra_clusters)}")
    log("   These will be ignored.")

log("✅ All clusters in the data are present in the mapping")
```

```
📋 ✅ All clusters in the data are present in the mapping
```

Apply Annotation to Dataset

Create the `cell_type_annotation` column by mapping Leiden clusters to cell types.

In [6]:

```
# Apply mapping to create cell_type_annotation column
adata.obs['cell_type_annotation'] =
adata.obs['leiden'].astype(str).map(mapping)

log("✅ Mapping applied: cell_type_annotation column created")

# Check for NaN values (indicates missing mapping)
n_missing = adata.obs['cell_type_annotation'].isna().sum()
if n_missing > 0:
    missing_clusters =
adata.obs[adata.obs['cell_type_annotation'].isna()]['leiden'].unique()
    raise ValueError(
        f"❌ ERROR: {n_missing} cells have missing cell_type_annotation
(NaN).\n"
```

```
        f"   Affected clusters: {missing_clusters}\n"
        f"   Please ensure all clusters in the mapping have valid cell type
labels."
    )

# Check for placeholder values
if "FILL_IN_MANUALLY" in adata.obs['cell_type_annotation'].values:
    n_placeholder = (adata.obs['cell_type_annotation'] ==
"FILL_IN_MANUALLY").sum()
    raise ValueError(
        f"❌ ERROR: {n_placeholder} cells still have placeholder annotation
('FILL_IN_MANUALLY').\n"
        f"   Please replace all placeholder values with actual cell type
names."
    )

log("✅ Validation passed: No missing or placeholder annotations")

# Display annotation summary
print("\n" + "="*70)
print("📊 ANNOTATION SUMMARY")
print("="*70)
print(f"Unique cell types annotated:
{adata.obs['cell_type_annotation'].nunique()}")
print(f"\nCell distribution by cell type:")
print(adata.obs['cell_type_annotation'].value_counts().sort_values(ascendin
g=False))
print("="*70 + "\n")
```

```
📋 ✅ Mapping applied: cell_type_annotation column created
📋 ✅ Validation passed: No missing or placeholder annotations

======================================================================
📊 ANNOTATION SUMMARY
======================================================================
Unique cell types annotated: 8

Cell distribution by cell type:
cell_type_annotation
Excitatory neuron          22769
Inhibitory neuron          13028
Oligodendrocyte/OPC         9696
Support/Vascular/Immune     6611
Microglia                   4875
Astrocyte Homeostatic       4752
Astrocyte Reactive           685
Unknown/Low Quality          384
Name: count, dtype: int64
======================================================================
```

Save Annotated Dataset

Save Updated AnnData Object

reduction_of_dataset → single_cell_pipeline → add_cell_type_annotation → translation_to_R

In [7]:

```
# OPTION 1 (RECOMMENDED): Overwrite adata_pp.h5ad
output_path = os.path.join(DIRS["DATA"], "adata_annotated.h5ad")
adata.write(output_path, compression='gzip')

log(f"✅ Annotated dataset saved (new file): {output_path}")
log(f"   File size: {os.path.getsize(output_path) / (1024**2):.2f} MB")
```

```
📋 ✅ Annotated dataset saved (new file):
C:/Z/AIDA_transcriptomics_project\data\adata_annotated.h5ad
📋    File size: 275.76 MB
```

Summary

In [8]:

```
#
==========================================================================
==
# VERIFICATION: PSEUDOBULK THRESHOLD VALIDATION
#
==========================================================================
==
# Objective: Validate that glial subtypes meet the minimum donor threshold
# Criteria: Min 5 donors with >= 20 cells per cluster (pseudobulk
requirement)
#
==========================================================================
==

MIN_CELLS_PER_SAMPLE = 20
MIN_DONORS_PER_CLUSTER = 5

print("="*80)
print("PSEUDOBULK THRESHOLD VALIDATION")
print("="*80)

# Function to verify cluster robustness
def verify_cluster(cluster_id, cluster_name):
    """
```

```
    Verify if a cluster passes the dual criteria:
    1. Total donors >= MIN_DONORS_PER_CLUSTER
    2. Donors with >= MIN_CELLS_PER_SAMPLE >= MIN_DONORS_PER_CLUSTER
    """
    # Filter cells from target cluster
    cluster_data = adata.obs[adata.obs['leiden'] == str(cluster_id)]

    # Count cells per donor
    cells_per_donor = cluster_data.groupby('donor_id').size()

    # Statistics
    n_total_cells = len(cluster_data)
    n_total_donors = cells_per_donor.shape[0]
    n_donors_above_threshold = (cells_per_donor >=
MIN_CELLS_PER_SAMPLE).sum()

    # Verdict
    pass_total_donors = n_total_donors >= MIN_DONORS_PER_CLUSTER
    pass_threshold_donors = n_donors_above_threshold >=
MIN_DONORS_PER_CLUSTER
    final_pass = pass_total_donors and pass_threshold_donors

    # Display results
    print(f"\n{'—'*80}")
    print(f"CLUSTER {cluster_id}: {cluster_name}")
    print(f"{'—'*80}")
    print(f"  Total cells           : {n_total_cells:,}")
    print(f"  Total donors          : {n_total_donors} {'[PASS]' if
pass_total_donors else '[FAIL]'} (threshold >= {MIN_DONORS_PER_CLUSTER})")
    print(f"  Donors >= {MIN_CELLS_PER_SAMPLE} cells :
{n_donors_above_threshold} {'[PASS]' if pass_threshold_donors else
'[FAIL]'} (threshold >= {MIN_DONORS_PER_CLUSTER})")

    # Distribution detail if failed
    if n_donors_above_threshold < MIN_DONORS_PER_CLUSTER:
        print(f"\n  WARNING: Unbalanced distribution detected")
        top_donors = cells_per_donor.nlargest(3)
        for donor, count in top_donors.items():
            pct = (count / n_total_cells) * 100
            print(f"     - {donor}: {count} cells ({pct:.1f}%)")

    print(f"\n  VERDICT: {'ROBUST' if final_pass else 'NON-VIABLE for
DGE'}")

    return {
        'cluster_id': cluster_id,
        'cluster_name': cluster_name,
        'n_cells': n_total_cells,
        'n_donors_total': n_total_donors,
```

```
        'n_donors_above_threshold': n_donors_above_threshold,
        'pass': final_pass
    }

# Verify critical glial clusters
results = []

print("\nASTROCYTES")
results.append(verify_cluster(4, "Astrocyte Homeostatic"))
results.append(verify_cluster(15, "Astrocyte Reactive"))

print("\nMICROGLIA")
results.append(verify_cluster(5, "Microglia Homeostatic"))
results.append(verify_cluster(18, "Microglia DAM/Activated"))

# Summary and architectural decision
print("\n" + "="*80)
print("ARCHITECTURAL DECISION SUMMARY")
print("="*80)

df_results = pd.DataFrame(results)
print("\n", df_results.to_string(index=False))

# Decision logic
astro_4_ok = df_results[(df_results['cluster_id'] == 4)]['pass'].values[0]
astro_15_ok = df_results[(df_results['cluster_id'] ==
15)]['pass'].values[0]
micro_5_ok = df_results[(df_results['cluster_id'] == 5)]['pass'].values[0]
micro_18_ok = df_results[(df_results['cluster_id'] ==
18)]['pass'].values[0]

print("\n" + "—"*80)
print("FINAL RECOMMENDATION:")
print("—"*80)

if astro_4_ok and astro_15_ok and micro_5_ok and micro_18_ok:
    print("\n[OPTION] 9 MACRO-TYPES")
    print("   All glial subtypes pass pseudobulk threshold.")

elif astro_4_ok and astro_15_ok and micro_5_ok and not micro_18_ok:
    print("\n[RECOMMENDED] 8 MACRO-TYPES")
    print("   - Astrocytes: Separation viable (Homeostatic vs Reactive)")
    print("   - Microglia: Merge required (Cluster 18 non-viable)")

elif not (astro_4_ok and astro_15_ok) and micro_5_ok and not micro_18_ok:
    print("\n[CONSERVATIVE] 7 MACRO-TYPES")
    print("   - Astrocytes and Microglia: Merge required")

else:
```

```
    print("\n[MANUAL REVIEW REQUIRED] Mixed configuration detected")

print("="*80)
================================================================================
=====
PSEUDOBULK THRESHOLD VALIDATION
================================================================================
=====

ASTROCYTES

--------------------------------------------------------------------------------
-----
CLUSTER 4: Astrocyte Homeostatic
--------------------------------------------------------------------------------
-----
  Total cells           : 4,752
  Total donors          : 17 [PASS] (threshold >= 5)
  Donors >= 20 cells : 9 [PASS] (threshold >= 5)

  VERDICT: ROBUST

--------------------------------------------------------------------------------
-----
CLUSTER 15: Astrocyte Reactive
--------------------------------------------------------------------------------
-----
  Total cells           : 685
  Total donors          : 17 [PASS] (threshold >= 5)
  Donors >= 20 cells : 7 [PASS] (threshold >= 5)

  VERDICT: ROBUST

MICROGLIA

--------------------------------------------------------------------------------
-----
CLUSTER 5: Microglia Homeostatic
--------------------------------------------------------------------------------
-----
  Total cells           : 4,482
  Total donors          : 17 [PASS] (threshold >= 5)
  Donors >= 20 cells : 14 [PASS] (threshold >= 5)

  VERDICT: ROBUST

--------------------------------------------------------------------------------
-----
CLUSTER 18: Microglia DAM/Activated
```

```
──────────────────────────────────────────────────────────────────────────────
─────
  Total cells           : 393
  Total donors          : 17 [PASS] (threshold >= 5)
  Donors >= 20 cells : 1 [FAIL] (threshold >= 5)

  WARNING: Unbalanced distribution detected
     - Donor_865: 386 cells (98.2%)
     - Donor_638: 2 cells (0.5%)
     - Donor_1467: 2 cells (0.5%)

  VERDICT: NON-VIABLE for DGE

================================================================================
=====
ARCHITECTURAL DECISION SUMMARY
================================================================================
=====

  cluster_id            cluster_name  n_cells  n_donors_total
n_donors_above_threshold  pass
          4   Astrocyte Homeostatic     4752              17
9  True
         15      Astrocyte Reactive      685              17
7  True
          5   Microglia Homeostatic     4482              17
14  True
         18 Microglia DAM/Activated      393              17
1 False

──────────────────────────────────────────────────────────────────────────────
─────
FINAL RECOMMENDATION:
──────────────────────────────────────────────────────────────────────────────
─────

[RECOMMENDED] 8 MACRO-TYPES
   - Astrocytes: Separation viable (Homeostatic vs Reactive)
   - Microglia: Merge required (Cluster 18 non-viable)
================================================================================
=====
```

In [9]:

```
# --- SCRIPT FINAL CORRIGÉ : ANNOTATION SUMMARY ---

# Remplacez le bloc existant par celui-ci pour résoudre la KeyError.

print("\n" + "="*80)
print("ANNOTATION SUMMARY")
print("="*80)
```

```
# 1. Détermination du nombre de clusters (utilisation de la colonne
'leiden')
# Nous utilisons 'leiden' comme valeur stable, vérifiez si 'leiden_0_5' est
le nom exact si l'erreur persiste.
cluster_col = 'leiden'

try:
    print(f"Total cells annotated: {adata.n_obs:,}")
    print(f"Total Leiden clusters: {adata.obs[cluster_col].nunique()}")
    print(f"Number of macro cell types:
{adata.obs['cell_type_annotation'].nunique()}") # Nom de colonne correct

    print(f"\nCell type distribution:")

    # 2. Affichage de la distribution par la colonne correcte
    for cell_type, count in
adata.obs['cell_type_annotation'].value_counts().sort_values(ascending=Fals
e).items():
        print(f"  {cell_type}: {count:,} cells
({count/adata.n_obs*100:.1f}%)")

except KeyError as e:
    # Ce bloc gère l'erreur et donne un diagnostic clair si la colonne est
introuvable.
    print(f"❌ ERREUR CRITIQUE D'AFFICHAGE : Clé '{e}' introuvable.")
    print("Veuillez vous assurer que la colonne 'cell_type_annotation' a
été créée à l'étape précédente.")

print(f"\n✅ ANNOTATION COMPLETE — ready for pseudobulk
(translation_to_R.ipynb)")
print("="*80 + "\n")
```

```
================================================================================
=====
ANNOTATION SUMMARY
================================================================================
=====
Total cells annotated: 62,800
Total Leiden clusters: 23
Number of macro cell types: 8

Cell type distribution:
  Excitatory neuron: 22,769 cells (36.3%)
  Inhibitory neuron: 13,028 cells (20.7%)
  Oligodendrocyte/OPC: 9,696 cells (15.4%)
  Support/Vascular/Immune: 6,611 cells (10.5%)
  Microglia: 4,875 cells (7.8%)
  Astrocyte Homeostatic: 4,752 cells (7.6%)
```

```
  Astrocyte Reactive: 685 cells (1.1%)
  Unknown/Low Quality: 384 cells (0.6%)

✅ ANNOTATION COMPLETE — ready for pseudobulk (translation_to_R.ipynb)
================================================================================
=====
```